

Checklist Servidor (DWES) v1.2 – Proyecto Final

Departamento de informática - IES Rafael Alberti

Índice de contenidos

Checklist DWES v1.2 – Proyecto Final (Unificada por stack)	1
API REST (70%)	1
Diseño impecable de recursos REST	1
Puntos de entrada bien organizados	2
Uso correcto de códigos HTTP	2
Autenticación y autorización con roles	2
Pruebas de API con buena cobertura	2
Documentación clara de la API	2
MVC (estructura del proyecto)	2
Separación de responsabilidades	2
Organización del proyecto por componentes	3
Autenticación y roles correctamente aplicados	3
Modelo de Datos (30%)	3
Modelo estructurado y bien relacionado	3
Consultas complejas y personalizadas	3
Definición de estructura de datos (migraciones o equivalente)	3
Documentación del modelo	3
Nota sobre la instalación y ejecución del servidor	4

Checklist DWES v1.2 – Proyecto Final (Unificada por stack)

Este documento unifica los criterios de evaluación por la rúbrica DWES v1.2, con anotaciones específicas para Laravel, Spring Boot, NestJS/Express o tecnologías equivalentes.

API REST (70%)

Diseño impecable de recursos REST

- ☐ Recursos bien definidos y separados por entidad (/users, /products, etc.)
- ☐ Convención RESTful respetada (GET, POST, PUT/PATCH, DELETE)
- ☐ Rutas limpias, sin verbos (/getUsers , /users)
- ☐ Uso correcto de rutas anidadas cuando corresponda (/users/{id}/orders)
- ☐ Soporte para paginación, filtros y ordenación (?page=1, ?sort=desc)
- ☐ Identificadores coherentes (id, uuid)
- ☐ Nombres de recursos en plural
- ☐ Documentación de todos los endpoints

Puntos de entrada bien organizados

- ☐ Controladores separados por dominio/lógica de negocio
- ☐ Rutas agrupadas y estructuradas por funcionalidad
- ☐ Uso de middlewares, interceptores o filtros según el stack
 - Laravel: Middleware + `routes/api.php`
 - Spring Boot: `@Controller`, `@RestController` + rutas REST
 - NestJS: `@Controller` decorado + Guards/Pipes
- ☐ Separación de responsabilidades clara: controladores no contienen lógica de negocio compleja

Uso correcto de códigos HTTP

- ☐ 200, 201, 204 para respuestas exitosas
- ☐ 400, 401, 403, 404, 422, 500 correctamente usados
- ☐ Mensajes de error estructurados (`{ error: ..., code: ... }`)
- ☐ Documentación específica códigos devueltos

Autenticación y autorización con roles

- ☐ Sistema de login funcionando (JWT, sesiones o token)
- ☐ Acceso a rutas protegido por middleware o mecanismo similar
- ☐ Gestión de roles:
 - Laravel: Middleware + `Policy`, `Gate`
 - Spring Boot: `@PreAuthorize`, `@Secured`, filtros de seguridad
 - NestJS: Guards + decoradores de roles
- ☐ Control de acceso efectivo a recursos según rol

Pruebas de API con buena cobertura

- ☐ Tests de los endpoints principales: éxito y error
- ☐ Autenticación probada (con/sin permisos)
- ☐ Validación del formato de respuesta JSON
- ☐ Tests automatizados:
 - Laravel: `Feature`, `PHPUnit`
 - Spring Boot: `MockMvc`, `WebMvcTest`
 - NestJS: `SuperTest`, `Jest`

Documentación clara de la API

- ☐ Documentación Swagger, OpenAPI o Postman
 - ☐ Ejemplos de uso con parámetros y respuestas
 - ☐ Explicación del sistema de autenticación
 - ☐ Repositorio incluye README con instrucciones de uso
-

MVC (estructura del proyecto)

Separación de responsabilidades

- ☐ Controladores gestionan la lógica de entrada/salida
- ☐ Lógica de negocio encapsulada en servicios o componentes
- ☐ Modelos acceden a la base de datos
- ☐ Las validaciones están separadas del controlador:
 - Laravel: `FormRequest`

- Spring Boot: `@Valid`, DTOs
- NestJS: DTOs + Pipes

Organización del proyecto por componentes

- ☐ Estructura clara por módulos o funcionalidades
- ☐ Archivos y carpetas organizadas (controladores, modelos, servicios...)
- ☐ Se evita duplicar código (uso de servicios compartidos)
- ☐ Se aplican patrones adecuados al stack (SRP, IoC, etc.)

Autenticación y roles correctamente aplicados

- ☐ Middleware, filtros o guards protegen rutas sensibles
 - ☐ Comprobaciones de rol en controladores o servicios
 - ☐ Comportamiento diferente según rol cuando aplica
-

Modelo de Datos (30%)

Modelo estructurado y bien relacionado

- ☐ Relación entre entidades definida y usada (1:1, 1:N, N:M)
- ☐ Definición clara de claves primarias y foráneas
- ☐ Uso de clases y anotaciones según stack:
 - Laravel: `hasMany`, `belongsTo`, `belongsToMany`
 - Spring Boot: `@OneToMany`, `@ManyToOne`, `@ManyToMany`
 - NestJS/TypeORM: `@OneToMany`, `@JoinTable`, etc.

Consultas complejas y personalizadas

- ☐ Consultas agregadas (`count`, `avg`, `sum`), filtros encadenados
- ☐ Queries anidadas o combinadas (`where`, `join`, `groupBy`)
- ☐ Consultas reutilizables:
 - Laravel: Scopes (`scopeActive()`)
 - Spring Boot: métodos en repositorios (`findByStatus()`)
 - NestJS: servicios o query builders reutilizables

Definición de estructura de datos (migraciones o equivalente)

- ☐ Laravel: migraciones (`php artisan migrate`), seeds
- ☐ Spring Boot: scripts SQL, Flyway o Liquibase
- ☐ NestJS: migraciones con TypeORM, Sequelize o Prisma
- ☐ Posibilidad de replicar la estructura sin tocar el código manualmente

Documentación del modelo

- ☐ Diagrama de entidad-relación (ER)
 - ☐ Descripción de tablas, campos y relaciones
 - ☐ Justificación del diseño (por qué se estructura así)
 - ☐ Se incluye en la memoria técnica del proyecto
-

Nota sobre la instalación y ejecución del servidor

Aunque el despliegue en producción se evalúa específicamente en el módulo **Despliegue de Aplicaciones Web**, en **DWES (Desarrollo Web en Entorno Servidor)** es imprescindible que el servidor pueda:

- Ser instalado y ejecutado correctamente en **entorno de desarrollo local**.
- Estar documentado con instrucciones claras (**README.md**, scripts de base de datos, etc.).
- Funcionar sin intervención del autor, solo siguiendo la documentación incluida.

Esto permite:

- Probar la API REST o la aplicación MVC.
- Ejecutar pruebas automáticas o manuales.
- Verificar el modelo de datos, autenticación y lógica de negocio.

Por tanto, **es obligatorio que el repositorio incluya todos los pasos necesarios para lanzar el servidor localmente** (instalación de dependencias, configuración, ejecución), independientemente de que también se despliegue en producción como parte del módulo correspondiente.
